

MODUL 9

PRAKTIKUM PEMROGRAMAN MULTIMEDIA & GAME

Procedural Maze Generation

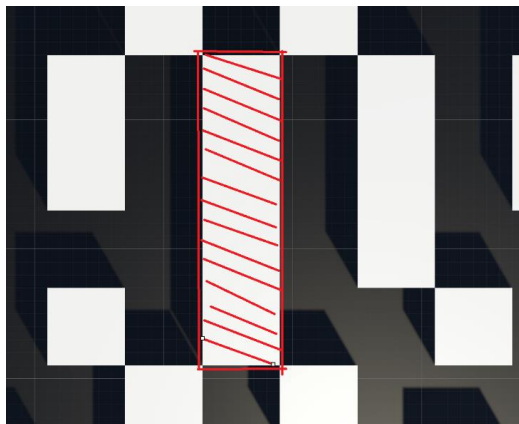
Counting Neighbours & Prims Algorithm

Pada Modul ini, Mahasiswa akan mempelajari tentang Maze Generation Prims Algorithm. Berikut adalah Langkah langkah implementasinya.

1. Defining Wall, Corridor and Room.

Pada pertemuan sebelumnya, sudah dijelaskan mengenai perbedaan Wall, dan corridor, dan pada pertemuan ini, akan dipelajari apakah itu Room/Ruangan dalam maze dan apakah perbedaanya dengan wall dan corridor.

- Wall & Corridor



- Room



2. Logika Counting Neighbours.

Counting Neighbours adalah metode untuk menghitung berapa banyak corridors/Lorong yang ada disekitar suatu object. Hal ini berfungsi agar AI mengetahui Langkah selanjutnya yang akan ditempuh agar jalur yang dilalui tetap menjadi Lorong. Jika tidak di implementasikan counting Neighbours, maka akan tergenerate banyak sekali Room yang tidak di inginkan.

Logika Counting Neighbours sama seperti penentuan koordinat suatu object dalam maze, dapat dilihat pada gambar dibawah ini.

$x-1, z+1$	$x, z+1$	$x+1, z+1$
$x-1, z$	x, z	$x+1, z$
$x-1, z-1$	$x, z-1$	$x+1, z-1$

Pada logika ini, hanya akan dihitung beberapa tetangga dengan sumbu horizontal dan vertical saja, karena sumbu diagonal akan membuat jalur diagonal yang tidak dapat dilewati oleh player.

3. Baris Kode Counting Neighbours.

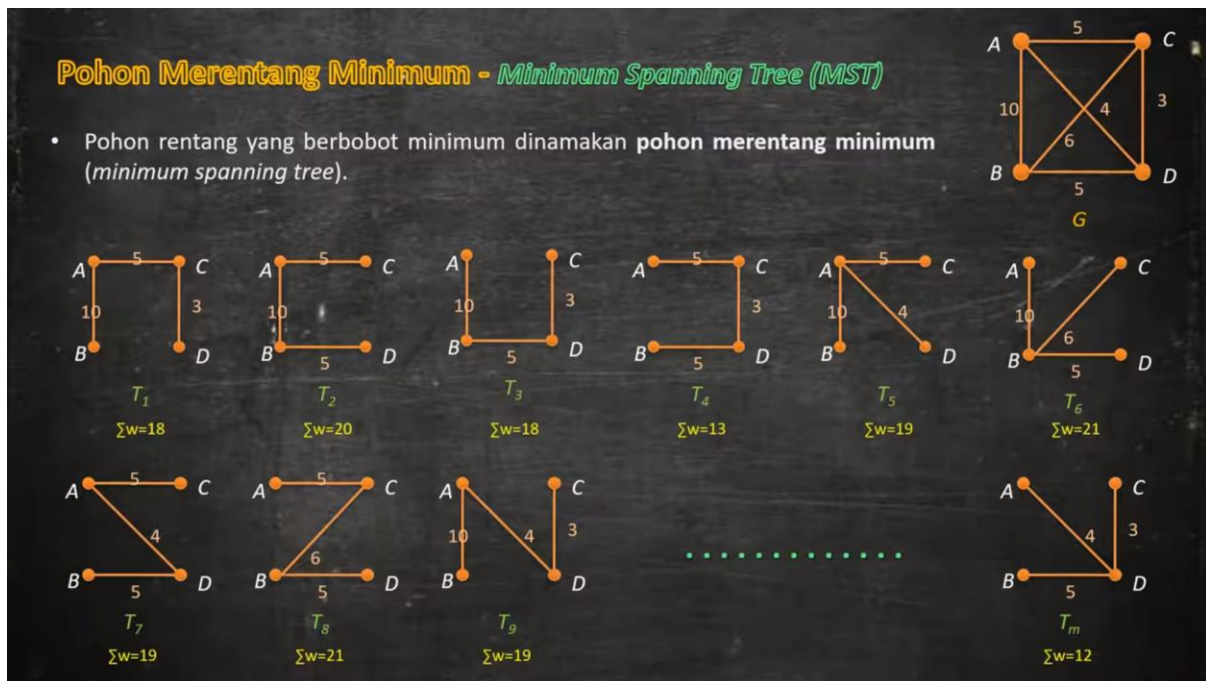
Implementasi Baris kode Counting Neighbours akan ditulis pada script **MazeLogic** sebagai berikut.

```
4 references
public int CountSquareNeighbours(int x, int z)
{
    int count = 0;
    if (x <= 0 || x >= width - 1 || z <= 0 || z >= depth - 1) return 5;
    if (map[x - 1, z] == 0) count++;
    if (map[x + 1, z] == 0) count++;
    if (map[x, z + 1] == 0) count++;
    if (map[x, z - 1] == 0) count++;
    return count;
}
```

Logika perhitungan tersebut akan ditampilkan pada **int count**, setiap koordinat **[x,z]** mendeteksi terdapat tetangga yang bersifat Lorong/koridor, maka **int count** akan bertambah 1. Dan jika koordinat tersebut mendeteksi tetangganya adalah ujung maze atau = **min size/max size**. Maka **int count** akan di return sebanyak 5.

4. Logika Prims Algoritma

Algoritma Prim adalah sebuah algoritma dalam teori graph untuk mencari pohon rentang minimum sebuah graph berbobot yang saling terhubung. Biasanya algoritma ini digunakan untuk mencari bobot terendah dari suatu jalur, tetapi algoritma tersebut juga dapat di implementasikan untuk generate jalur Maze.



5. Implementasi Prims Algoritma pada Unity

Berikut adalah Langkah Langkah implementasi Prims Algoritma pada unity.

- Buat Script baru Bernama **PrimsAlgoritma** didalam GameObject **MazeManager**.
- Pada Script **PrimsAlgoritma** akan mengimplementasi **Inheritance** dari script **MazeLogic**

```
Unity Script | 0 references
public class CrawlerAlgoritma : MazeLogic
{
    ...
}
```

- Kembali ke Script **MazeLogic**, Tulis Class Baru Bernama MapLocation di atas Class MazeLogic seperti pada gambar dibawah ini. fungsi Class tersebut adalah untuk menampung value X dan Y.

```

11 references
public class MapLocation
{
    public int x;
    public int z;

    8 references
    public MapLocation(int _x, int _z)
    {
        x = _x;
        z = _z;
    }
}

@ Unity Script | 2 references
public class MazeLogic : MonoBehaviour
{
    public int width = 30; //x length
    public int depth = 30; //z length
    public List<GameObject> Cube; //Maze Wall
    public byte[,] map;
}

```

- Tulis Beberapa Baris Script dibawah, dan hapus Method **Start** dan **Update**.

```

2 references
public override void GenerateMaps()
{
    int x = 2;
    int z = 2;

    map[x, z] = 0;

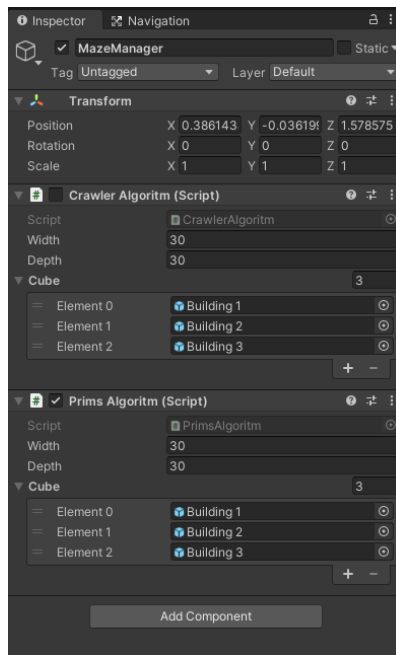
    List<MapLocation> walls = new List<MapLocation>();
    walls.Add(new MapLocation(x + 1, z));
    walls.Add(new MapLocation(x - 1, z));
    walls.Add(new MapLocation(x, z + 1));
    walls.Add(new MapLocation(x, z - 1));

    int countloops = 0;
    while (walls.Count > 0 && countloops < 5000)
    {
        int rwall = Random.Range(0, walls.Count);
        x = walls[rwall].x;
        z = walls[rwall].z;
        walls.RemoveAt(rwall);
        if (CountSquareNeighbours(x, z) == 1)
        {
            map[x, z] = 0;
            walls.Add(new MapLocation(x + 1, z));
            walls.Add(new MapLocation(x - 1, z));
            walls.Add(new MapLocation(x, z + 1));
            walls.Add(new MapLocation(x, z - 1));
        }

        countloops++;
    }
}

```

- Inisialisasi seperti gambar dibawah, aktifkan Prims Algoritm dan nonaktifkan CrawlerAlgoritm.



- Running Program, dan amati Maze yang tergenerate. akan terbentuk pola yang lebih terarah namun tetap tergenerate secara random. Pola yang akan terbentuk kurang lebihnya seperti pada gambar dibawah, jika pola masih belum sama, lakukan pengecekan ulang!.

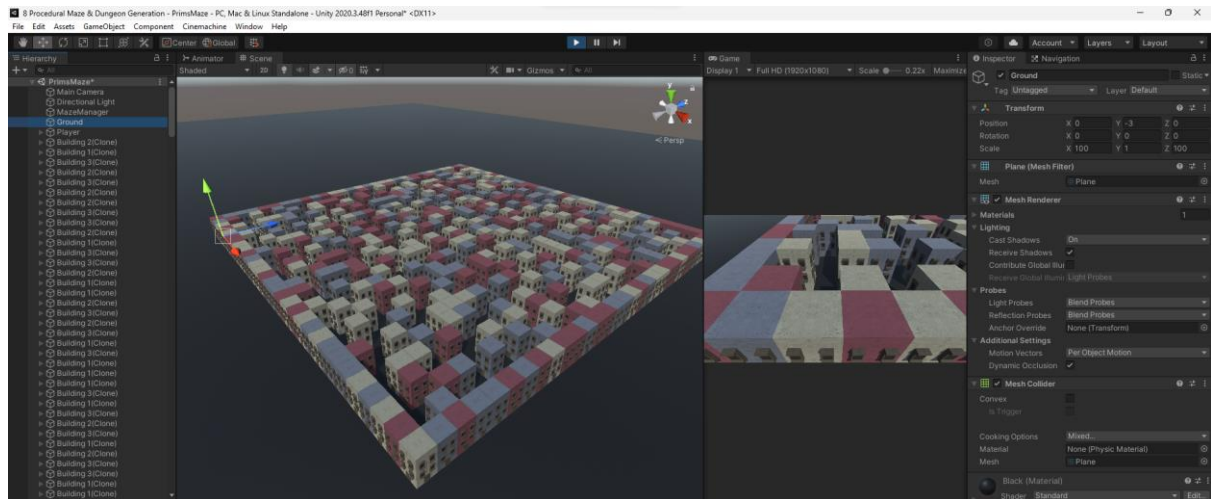


6. Scale Up Environment and Why.

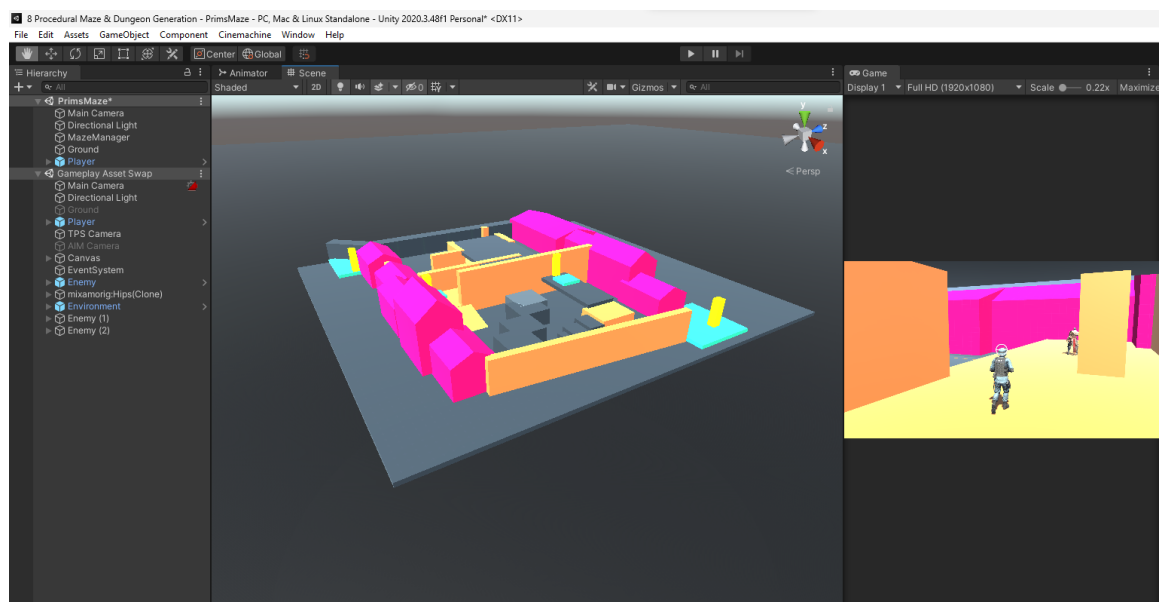
Scale Up Environment ini berfungsi untuk mengatur skala environment, hal ini karena setiap karakter memiliki skala yang berbeda beda, dan setiap scenario juga

memiliki latar tempat yang berbeda, oleh karena itu proses customisasi skala perlu dilakukan. Berikut adalah Langkah Langkah customisasi skala.

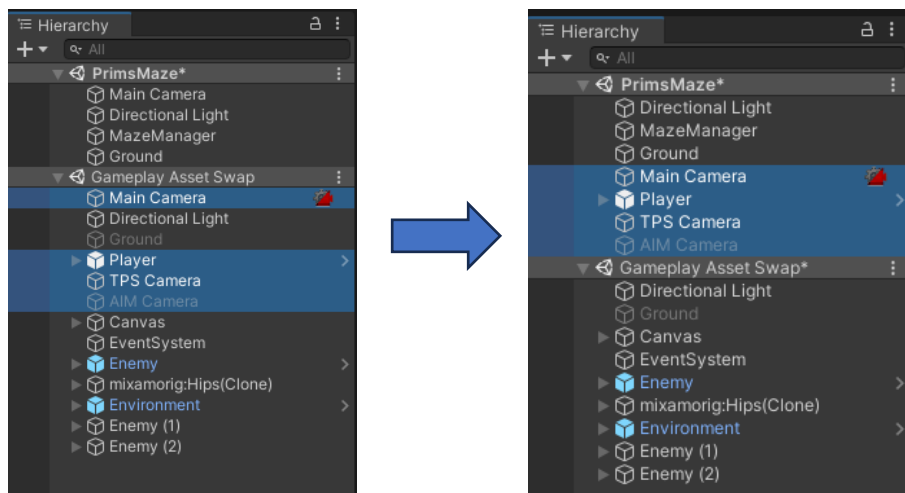
- Create Ground Dari GameObject Plane berfungsi sebagai alas. Atur Material sesuai yang diinginkan.



- Import Karakter prefabs kedalam Hierarchy dengan cara import scene gameplay.

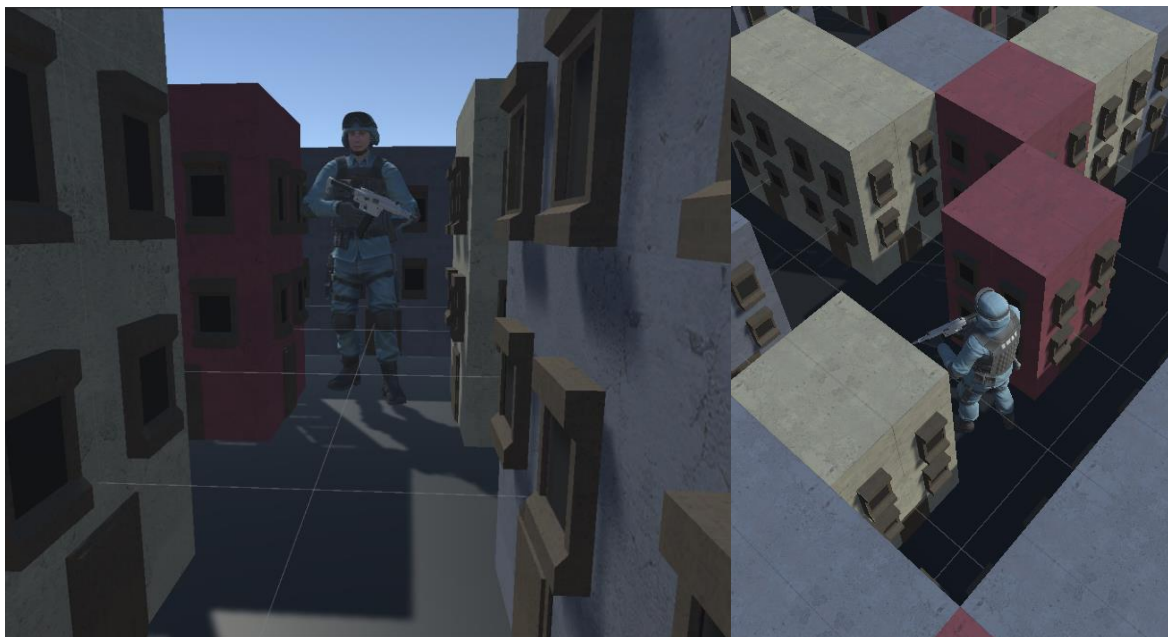


- Drag and Drop Player, Main Camera, TPS Camera, AIM Camera, dan Canvas kedalam Scene Maze.



Delete Main Camera Yang berada dialam Scene Maze. Lalu Remove Scene Gameplay dengan Klik kanan Scene tersebut, pilih Remove Scene, dan pilih Don't Save Agar Scene Gameplay tidak ada perubahan.

- masuk kedalam Play Mode. Dan amati Skala Object.



Pada tampilan di atas akan terlihat jika skala karakter terlihat sangat besar, ini menyebabkan tampilan object tidak realistis. Oleh karena itu proses skala Environment sangat dibutuhkan.

Buka Script Maze Logic dan tulisan beberapa baris kode berikut.

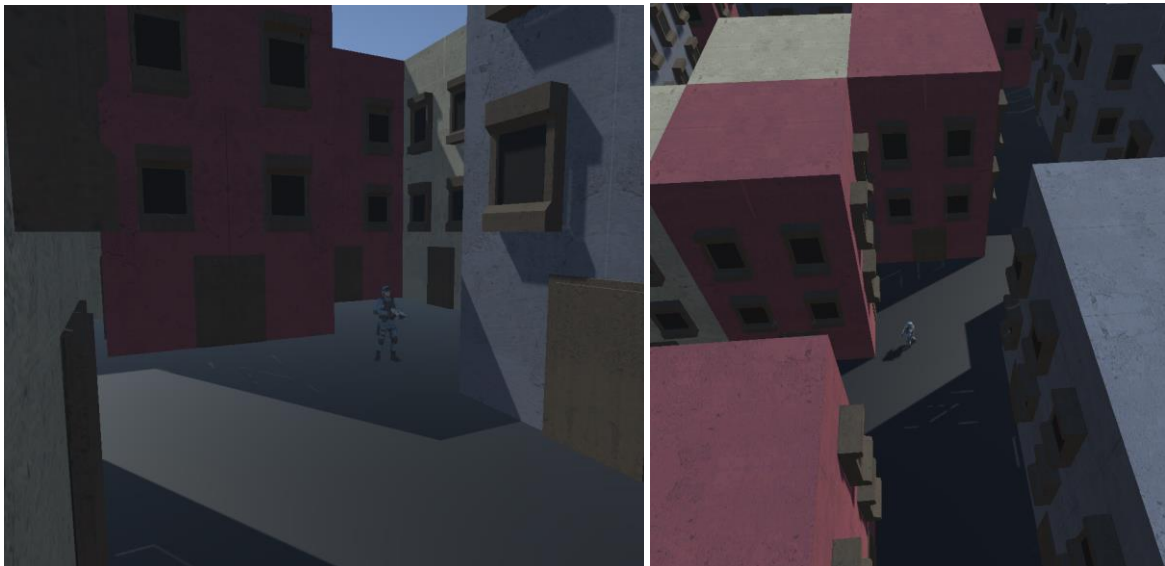
- Tambahkan 1 Variable Global baru.

```
Unity Script | 2 references
public class MazeLogic : MonoBehaviour
{
    public int width = 30; //x length
    public int depth = 30; //z length
    public int scale = 6;
    public List<GameObject> Cube; //Maze Wall
    public byte[,] map;
```

- Sesuaikan Method DrawMaps

```
1 reference
void DrawMaps() // Draw All Value With GameObject
{
    for (int z = 0; z < depth; z++)
        for (int x = 0; x < width; x++)
        {
            if (map[x, z] == 1)
            {
                Vector3 pos = new Vector3(x * scale, 0, z * scale);
                GameObject wall = Instantiate(Cube[Random.Range(0, Cube.Count)], pos, Quaternion.identity);
                wall.transform.localScale = new Vector3(scale, scale, scale);
                wall.transform.position = pos;
            }
        }
}
```

- Save, Kembali ke unity dan Masuk kedalam Play mode.



Pada Gambar di atas terlihat Tampilan Environment yang sudah di scale up, terlihat tampilan berbandingan skala antara karakter dan Gedung lebih proporsional.

7. Place Characters on the Maze Automatically with Script.

Proses Selanjutnya adalah memasang posisi karakter secara otomatis melalui Script. Logikanya adalah player akan diposisikan secara random didalam Maze Lorong maze, berikut adalah script implementasinya.

- Tulis Variable Global Baru

```
public int width = 15; //x length
public int depth = 15; //z length
public byte[,] map;
public int scale = 6;
public GameObject Character;
```

- Tulis Method Baru Bernama PlaceCharacter.

```
1 reference
public virtual void PlaceCharacter()
{
    bool PlayerSet = false;
    for (int i = 0; i < depth; i++)
    {
        for (int j = 0; j < width; j++)
        {
            int x = Random.Range(0, width);
            int z = Random.Range(0, width);
            if (map[x, z] == 0 && !PlayerSet)
            {
                Debug.Log("placing character");
                PlayerSet = true;
                Character.transform.position = new Vector3(x * scale, 0, z * scale);
            }
            else if (PlayerSet)
            {
                Debug.Log("already Placing character");
                return;
            }
        }
    }
}
```

- Sesuaikan Method Start

```
Unity Message | 0 references
void Start()
{
    InitialiseMap();
    GenerateMaps();
    DrawMaps();
    PlaceCharacter();
}
```

Kembali Ke unity dan masuk kedalam Play mode. Jika karakter sudah berpindah posisi secara random, dan posisi karakter sudah selalu berada pada Lorong maze, maka konfigurasi dan script sudah benar, jika belum lakukan pengecekan ulang!

8. Selesai Untuk Praktikum 9

Praktikum 9 Dikatakan Selesai Jika Praktikan dapat mengimplementasikan Maze Generation menggunakan algoritma Prim's Algorithm dan memasang karakter didalam labirin.

Tugas Modul 9

Tidak ada Tugas Untuk Modul Ke 9

Fokus kepada progressing ke 4 dan maju secara berkelompok ke dosen pengampu mata kuliah praktikum game.